

Manuel technique utilisateur

Logiciel de Corrélation d'Images Numériques 2D

DIC_2D_V3.1

Digital Image Correlation — Pipeline d'analyse 2D

Document technique correspondant au paquet `run_dic_v3.py` /
DIC_2D_V3

Micro-certification — Essai pilote
Beca Santander – FIMPES Recherche

Table des matières

1	Introduction et portée du document	3
2	Fondements conceptuels implémentés	3
2.1	Hypothèse de conservation de l'intensité	3
2.2	Critère de corrélation : ZNCC	3
2.3	Raffinement sous-pixel par interpolation parabolique	4
2.4	Mode de corrélation incrémental	4
2.5	Calcul de la déformation : gradients de déplacement par MCL locale	5
2.6	Propagation de l'incertitude (mode GUM)	5
3	Structure du paquet	6
4	Paramètres d'entrée du script principal	7
4.1	Bloc PROJECT : chemins et nom du projet	7
4.2	Bloc COMPUTE : calcul et accélération	8
4.3	Bloc IMAGING : image et étalonnage	9
4.4	Bloc ROI : région d'intérêt et grille d'analyse	10
4.5	Bloc DIC : paramètres du moteur de corrélation	11
4.6	Bloc DISPLACEMENT : post-traitement du déplacement	12
4.7	Bloc STRAIN : déformation	13
4.8	Bloc UNCERTAINTY : incertitude	13
4.9	Bloc VIS : visualisation et export	14
4.10	Bloc RUNTIME : contrôle d'exécution	15
5	Description technique des modules	15
5.1	Chargement des images : image_loader.py	15
5.2	Étalonnage : calibration.py	16
5.3	ROI et grille : roi_grid.py	17
5.4	Moteur de corrélation : correlation_engine.py	17
5.4.1	Troncature de la queue de corrélation	18
5.5	Post-traitement du déplacement : displacement.py	19
5.6	Calcul de la déformation : strain.py	19
5.7	Incertainitude : uncertainty.py	19
6	Exécution du pipeline	20
6.1	Prérequis système	20
6.2	Structure de répertoires attendue	20
6.3	Exécution standard	21
7	Interprétation des sorties	21
7.1	Tableau de bord global	21
7.2	Champ vectoriel de déplacement (quiver)	22
7.3	Cartes individuelles de déformation	23
7.4	Évolution temporelle de la déformation	24

7.5	Déformation vs. déplacement maximal	25
7.6	Résumé d'incertitude	25
7.7	Tableau GUM	26
8	Sorties de l'image sélectionnée	26
9	Fichiers générés par le pipeline	29
9.1	Fichiers d'étalonnage	29
9.2	Fichiers de données numériques	29
9.2.1	Déformation	29
9.2.2	Incertainitude	29
9.3	État du projet	29
10	Critères de configuration recommandés	29
11	Limitations de la version actuelle	30

1 Introduction et portée du document

Ce manuel décrit le fonctionnement technique du logiciel **DIC_2D_V3.1**, un pipeline modulaire en Python pour la Corrélation d’Images Numériques en deux dimensions (DIC 2D). Le logiciel a été développé dans le cadre de la Micro-certification d’Analyse des Contraintes par DIC, Essai pilote, sous la Beca Santander–FIMPES Recherche.

Le présent document est complémentaire au *Manuel technique du participant*. Tandis que ce manuel établit les fondements théoriques de la DIC (critères de corrélation, subsets, étalonnage, tenseur de déformation, incertitude GUM), le présent manuel décrit *comment ces concepts sont concrètement implémentés* dans le code DIC_2D_V3.1, quels paramètres contrôlent chaque étape et comment interpréter les sorties numériques et graphiques générées.

L’orchestrateur principal est `run_dic_v3.py`. Tous les paramètres de configuration sont centralisés au début de ce script. L’utilisateur n’a pas besoin de modifier les modules du paquet DIC_2D_V3/ pour une exécution standard.

2 Fondements conceptuels implémentés

Lien avec le Manuel du participant

Cette section résume les concepts développés en détail mathématique dans les chapitres 1, 3 et 4 du *Manuel technique du participant*. Il est recommandé de consulter ce document avant d’exécuter le pipeline pour la première fois.

2.1 Hypothèse de conservation de l’intensité

Le principe fondamental de la DIC est que l’intensité de l’image est conservée entre l’image de référence $f(\mathbf{x})$ et l’image déformée $g(\mathbf{x})$ sous la transformation du subset :

$$f(\mathbf{x}) \approx g(\mathbf{x} + \mathbf{u}(\mathbf{x})), \quad (1)$$

où $\mathbf{u} = (u, v)$ est le champ de déplacement dans le plan de l’image. L’objectif de la corrélation est de trouver $\mathbf{u}$ qui minimise l’écart entre le subset de référence et le subset déformé.

2.2 Critère de corrélation : ZNCC

Dans `correlation_engine.py`, le noyau de recherche utilise la fonction `cv2.matchTemplate` avec le critère `TM_CCOEFF_NORMED`, qui est numériquement équivalent au coefficient de corrélation croisée normalisé avec soustraction de la moyenne (ZNCC, *Zero-Normalized Cross*

Correlation) :

$$C_{\text{ZNCC}}(u, v) = \frac{\sum_{(\xi, \eta) \in S} [f(\xi, \eta) - \bar{f}] [g(\xi + u, \eta + v) - \bar{g}_{(u, v)}]}{\sqrt{\sum_{(\xi, \eta) \in S} [f(\xi, \eta) - \bar{f}]^2 \sum_{(\xi, \eta) \in S} [g(\xi + u, \eta + v) - \bar{g}_{(u, v)}]^2}}, \quad (2)$$

où S est le domaine du subset de rayon r (`subset_radius`), $\bar{f}$ et $\bar{g}_{(u, v)}$ sont les moyennes locales d'intensité. Ce critère est invariant aux variations d'offset et de gain d'éclairement, ce qui le rend robuste face aux variations d'illumination entre images. La plage de C_{ZNCC} est $[-1, 1]$; le paramètre `corrcoef_min` définit le seuil d'acceptation minimal.

2.3 Raffinement sous-pixel par interpolation parabolique

Une fois le maximum entier de la carte de corrélation identifié au pixel $(i_{\text{max}}, j_{\text{max}})$, un ajustement parabolique unidimensionnel est appliqué dans chaque direction pour estimer la position sous-pixel du pic. Pour la direction x :

$$\delta_x = \frac{C(i_{\text{max}} - 1) - C(i_{\text{max}} + 1)}{2[C(i_{\text{max}} - 1) - 2C(i_{\text{max}}) + C(i_{\text{max}} + 1)]}, \quad (3)$$

et de manière analogue pour y . Le déplacement sous-pixel δ_x est ajouté à la position entière du maximum. Ce schéma est implémenté dans la fonction `_parabolic_subpixel_1d()` et correspond à la méthode d'interpolation parabolique du pic décrite dans le Manuel du participant (§1.6).

2.4 Mode de corrélation incrémental

Le paramètre `mode = "incremental"` active le mode de corrélation pas à pas (*step analysis*), analogue à la stratégie de suivi matériel de NCorr. Dans ce mode, pour chaque paire d'images consécutives $(k - 1, k)$:

1. Le template est extrait autour de la position *matérielle* du point à la fin de l'image précédente : $(x_0 + u_{k-1}, y_0 + v_{k-1})$.
2. Un prédicteur basé sur l'incrément précédent est appliqué : la fenêtre de recherche est centrée sur $(x_0 + u_{k-1} + \Delta u_{k-1}, y_0 + v_{k-1} + \Delta v_{k-1})$.
3. L'incrément mesuré $(\Delta u_k, \Delta v_k)$ est accumulé pour obtenir le déplacement total par rapport à la référence : $u_k = u_{k-1} + \Delta u_k$.

Ce mécanisme est essentiel pour les essais à grands déplacements, où la corrélation directe image-référence échouerait par perte du motif de mouchetis à l'intérieur du subset d'origine.

2.5 Calcul de la déformation : gradients de déplacement par MCL locale

Le module `strain.py` implémente la méthode `ncorr_dispgrad`, qui approxime l'approche de la *jauge de déformation virtuelle* (VSG) de NCorr. À chaque nœud de la grille (i, j) , un plan du premier ordre est ajusté localement sur un voisinage circulaire de rayon `strain_radius` r :

$$u(x, y) \approx a_0 + a_1 x + a_2 y, \quad v(x, y) \approx b_0 + b_1 x + b_2 y, \quad (4)$$

en résolvant le système aux moindres carrés :

$$\min_{\mathbf{a}} \|\mathbf{A}\mathbf{a} - \mathbf{u}_{\text{local}}\|^2, \quad \mathbf{A} = \begin{bmatrix} 1 & x_1 & y_1 \\ \vdots & \vdots & \vdots \\ 1 & x_N & y_N \end{bmatrix}, \quad (5)$$

où N est le nombre de nœuds à l'intérieur du voisinage circulaire. Les gradients de déplacement extraits sont $\partial u / \partial x = a_1$, $\partial u / \partial y = a_2$, $\partial v / \partial x = b_1$, $\partial v / \partial y = b_2$.

Pour le tenseur de déformation infinitésimale (option active par défaut) :

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \quad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \quad \varepsilon_{xy} = \frac{1}{2} \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right). \quad (6)$$

2.6 Propagation de l'incertitude (mode GUM)

Le module `uncertainty.py` implémente une propagation GUM simplifiée. L'incertitude de base du déplacement en pixels σ_{px} est estimée à partir de l'erreur de reprojection de l'étalonnage (si disponible) ou bien une valeur conservatrice de 0.05 px est assignée pour la DIC 2D sous-pixel :

$$\sigma_{\text{disp}}(i, k) = \sigma_{\text{px}} \cdot p(C_{\text{ZNCC}}(i, k)), \quad (7)$$

où $p(C)$ est un facteur de pénalisation qui augmente l'incertitude lorsque le coefficient de corrélation s'éloigne de l'unité :

$$p(C) = 1 + \text{clip} \left(\frac{1 - C}{0.10}, 0, 5 \right). \quad (8)$$

L'incertitude combinée du vecteur de déplacement est :

$$u_c = \sqrt{\sigma_x^2 + \sigma_y^2}. \quad (9)$$

La propagation à la déformation considère que chaque composante est essentiellement une dérivée numérique sur la longueur effective de la jauge virtuelle $L_x = 2r \Delta_x$ et $L_y = 2r \Delta_y$:

$$u(\varepsilon_{xx}) = \frac{\sqrt{2} \sigma_u}{L_x}, \quad u(\varepsilon_{yy}) = \frac{\sqrt{2} \sigma_v}{L_y}, \quad u(\varepsilon_{xy}) = \frac{1}{2} \sqrt{\left(\frac{\sqrt{2} \sigma_u}{L_y} \right)^2 + \left(\frac{\sqrt{2} \sigma_v}{L_x} \right)^2}. \quad (10)$$

3 Structure du paquet

Le paquet distribué dans `DIC_2D_V3_1_mejorado.zip` contient les fichiers présentés dans le Tableau 1.

Fichier	Rôle dans le pipeline
<code>run_dic_V3.py</code>	Orchestrateur principal. Définit tous les paramètres et exécute la chaîne complète d'analyse.
<code>DIC_2D_V3/config.py</code>	Classes de configuration : <code>DICConfigV3</code> et sous-classes par responsabilité (<code>ComputeConfig</code> , <code>ImagingConfig</code> , <code>ROIConfig</code> , etc.).
<code>DIC_2D_V3/project.py</code>	Classe <code>DICProjectV3</code> : conteneur de l'état du projet et de toutes les matrices de résultats.
<code>DIC_2D_V3/image_loader.py</code>	Chargement des images, validation de la résolution commune et normalisation en <code>float32</code> en niveaux de gris.
<code>DIC_2D_V3/calibration.py</code>	Métrologie d'échelle mm/px (interactive ou depuis un fichier), correction de la distorsion optique et persistance de l'étalonnage.
<code>DIC_2D_V3/roi_grid.py</code>	Définition de la ROI rectangulaire interactive ou programmatique; génération de la grille réduite de points d'analyse.
<code>DIC_2D_V3/correlation_engine.py</code>	Moteur de corrélation ZNCC avec suivi incrémental, prédicteur local, raffinement sous-pixel parabolique et repli absolu.
<code>DIC_2D_V3/displacement.py</code>	Filtrage des valeurs aberrantes, comblement des lacunes, lissage spatial et construction des champs de déplacement en px et mm.
<code>DIC_2D_V3/strain.py</code>	Gradients de déplacement par MCL locale (<code>dispgad</code> type <code>NCorr</code>) et calcul des tenseurs de déformation.
<code>DIC_2D_V3/uncertainty.py</code>	Estimation de l'incertitude GUM pour le déplacement et la déformation avec propagation analytique.
<code>DIC_2D_V3/visualization.py</code>	Tableau de bord des résultats, champ vectoriel (quiver), cartes de champs scalaires, courbes temporelles et tableau GUM.
<code>DIC_2D_V3/utils.py</code>	Chronomètre, configuration du logging et fonctions auxiliaires partagées.

TAB. 1 : Structure fonctionnelle du paquet `DIC_2D_V3`.

Important

Le fichier d'initialisation du paquet est nommé `DIC_2D_V3__init__.py` dans le ZIP distribué. Pour que Python le reconnaisse correctement comme `__init__.py` du paquet, il doit être renommé en `__init__.py` dans le répertoire `DIC_2D_V3/` avant d'exécuter le pipeline.

4 Paramètres d'entrée du script principal

Tous les paramètres de configuration se trouvent dans les dictionnaires au début de `run_dic_v3.py`, dans la section marquée par le commentaire `# PARÁMETROS DE ENTRADA - EDITAR AQUÍ`. L'utilisateur doit éditer uniquement cette section. Les modules du paquet `DIC_2D_V3/` ne doivent pas être modifiés en fonctionnement normal.

4.1 Bloc PROJECT : chemins et nom du projet

Paramètre	Valeur par défaut	Description
<code>name</code>	<code>"pw-0mil-p2-n14"</code>	Identifiant du projet. Utilisé pour nommer le fichier d'état <code>.dicv3</code> et les fichiers d'étalonnage.
<code>image_dir</code>	<code>.\imagenes</code>	Répertoire contenant la séquence d'images de l'essai.
<code>image_pattern</code>	<code>"pw-0mil-p2-n14*.tif"</code>	Motif glob pour sélectionner les fichiers image. L'ordre de chargement suit l'ordre alphanumérique des noms.
<code>output_dir</code>	<code>.\resultados_dic_v3</code>	Répertoire de destination pour toutes les sorties : figures, fichiers de données et état du projet.

4.2 Bloc COMPUTE : calcul et accélération

Paramètre	Valeur par défaut	Description
use_gpu	True	Active la détection du GPU via PyTorch.
gpu_backend	"torch"	Backend d'accélération. Options : "torch", "none".
gpu_device	"auto"	Sélection du dispositif : "auto" choisit CUDA s'il est disponible, "cpu" force le CPU.
gpu_for_correlation	False	La corrélation s'exécute sur CPU dans cette version. Le GPU ne participe pas au noyau de matching.
gpu_for_visualization	True	Le rééchantillonnage des champs pour la visualisation peut exploiter le GPU via PyTorch.
num_threads	0	Nombre de threads OpenCV/NumPy. La valeur 0 sélectionne automatiquement le nombre optimal.

Note technique

La corrélation DIC dans cette version (V3.1) s'exécute intégralement sur CPU avec des opérations OpenCV et NumPy. L'indicateur `gpu_for_correlation = False` est correct et ne doit pas être modifié à `True` dans cette version, car la logique de suivi incrémental n'a pas d'implémentation GPU.

4.3 Bloc IMAGING : image et étalonnage

Paramètre	Valeur par défaut	Description
<code>apply_calibration</code>	True	Active l'étape d'étalonnage/métrologie. Si False, tous les résultats restent en pixels.
<code>interactive_calibration</code>	True	Demande à l'utilisateur de sélectionner deux points sur l'image de référence et de saisir la distance réelle en mm.
<code>use_saved_calibration_if_available</code>	True	Si un fichier d'étalonnage antérieur existe (.npz ou .json), il est réutilisé sans solliciter d'interaction.
<code>distortion_correction</code>	True	Applique la correction de distorsion radiale si le fichier d'étalonnage contient les paramètres intrinsèques de la caméra (<code>camera_matrix</code> , <code>dist_coeffs</code>).
<code>output_units</code>	"mm"	Unités de sortie pour les champs de déplacement : "mm" nécessite un étalonnage valide; "px" non.
<code>strain_display_units</code>	"microstrain"	Échelle d'affichage de la déformation. Le calcul interne est toujours adimensionnel; la conversion en $\mu\epsilon$ est appliquée uniquement dans les figures et les fichiers d'export secondaires.

4.4 Bloc ROI : région d'intérêt et grille d'analyse

Paramètre	Valeur par défaut	Description
interactive	True	Si True, la ROI est sélectionnée manuellement sur l'image de référence en cliquant sur deux coins opposés.
type	"rectangular"	Type de ROI. Dans cette version, l'option active est "rectangular". Le module supporte la ROI polygonale programmatique, mais ce n'est pas le flux par défaut.
spacing_x	15	Espacement horizontal entre centres de subset, en pixels. Détermine la densité de la grille d'analyse en x .
spacing_y	15	Espacement vertical entre centres de subset, en pixels. Détermine la densité de la grille d'analyse en y .

Lien avec le Manuel du participant

La relation entre l'espacement de la grille Δ , le rayon du subset r et la résolution spatiale de la déformation est analysée en détail aux §4.3.2 et §4.3.3 du Manuel du participant. La *jauge de déformation virtuelle* (VSG) effective a pour longueur $L = 2r_s \Delta$, où r_s est le `strain_radius`.

4.5 Bloc DIC : paramètres du moteur de corrélation

Paramètre	Valeur par défaut	Description
subset_radius	21	Rayon du subset en pixels. Le subset a pour dimension $(2r + 1) \times (2r + 1) = 43 \times 43$ px. Des subsets plus grands augmentent la résistance au bruit mais réduisent la résolution spatiale.
subset_spacing	15	Pas de la grille. Assigné à <code>cfg.dic.spacing</code> . Doit coïncider avec <code>ROI["spacing_x"]</code> et <code>ROI["spacing_y"]</code> pour la cohérence.
cutoff_diffnorm	1e-6	Critère de convergence pour le raffinement itératif (non actif dans le moteur NCC; réservé aux implémentations IC-GN futures).
cutoff_iteration	50	Nombre maximal d'itérations autorisées (idem ci-dessus).
step_analysis	True	Active la corrélation incrémentale image par image. Voir §2.4.
corrcoef_min	0.50	Seuil minimal du coefficient ZNCC pour accepter un point comme valide. Les valeurs inférieures indiquent une corrélation peu fiable.
search_radius	23	Rayon de la fenêtre de recherche autour de la position prédite. Doit être supérieur au déplacement maximal entre images consécutives.
mode	"incremental"	Mode de corrélation : "incremental" = image par image cumulé; "absolute" = toutes les images contre la référence.
seed_strategy	"propagation"	Stratégie de propagation des graines pour l'initialisation de la corrélation.
use_predictor	True	Active le prédicteur basé sur l'incrément de l'image précédente. Améliore le taux de réussite lors d'essais à grandes déformations.
engine	"rgdic"	Moteur déclaré. En V3.1, le matching NCC est le noyau effectif.
interpolation	"bicubic"	Interpolation pour l'extraction sous-pixel du template : "bicubic" $\rightarrow$ <code>cv2.INTER_CUBIC</code> , "bilinear" $\rightarrow$ <code>cv2.INTER_LINEAR</code> .
warp	"affine"	Modèle de déformation locale du subset. En V3.1, la translation pure est utilisée; le champ "affine" est déclaratif.

Recommandation opérationnelle

Pour choisir `subset_radius`, le critère pratique est que le subset doit contenir au moins 3 à 5 taches du mouchetis. Si le speckle a une taille typique de 5–8 px, un rayon de 15–25 px est adéquat. Un rayon excessif produit un lissage spatial ; un rayon insuffisant produit des corrélations bruitées.

4.6 Bloc DISPLACEMENT : post-traitement du déplacement

Paramètre	Valeur par défaut	Description
<code>outlier_method</code>	"corrcoef+median"	Méthode de détection des points aberrants. Combine un seuillage par coefficient de corrélation et un filtre médian local sur les déplacements.
<code>median_threshold</code>	2.5	Seuil en écarts-types robustes pour la détection des valeurs aberrantes par médiane. Les points qui s'écartent de plus de ce seuil sont rejetés.
<code>spatial_smoothing</code>	"gaussian"	Lissage spatial des champs de déplacement post-traités. Options: "none", "gaussian", "median".
<code>smoothing_kernel</code>	3	Taille du noyau de lissage en unités de nœuds de grille.
<code>fill_missing</code>	True	Comble les nœuds invalides par interpolation sur les voisins valides.
<code>fill_method</code>	"griddata"	Méthode d'interpolation spatiale pour le comblement des lacunes. "griddata" utilise une interpolation triangulaire irrégulière via Delaunay.
<code>export_displacements_in_mm</code>	True	Génère les versions métriques (<code>disp_x_mm</code> , <code>disp_y_mm</code> , <code>disp_mag_mm</code>) lorsque <code>mm_per_px</code> est valide.

4.7 Bloc STRAIN : déformation

Paramètre	Valeur par défaut	Description
engine	"ncorr_dispgrad"	Algorithme de calcul des gradients de déplacement. La méthode active approxime le dispgrad de NCorr : ajustement MCL linéaire local sur voisinage circulaire. Alternative : "gradient" (différences finies via <code>numpy.gradient</code>).
strain_radius	7	Rayon du voisinage circulaire sur la grille réduite pour l'ajustement MCL. Définit la taille du VSG effectif : $L = 2 \times 7 \times 15 = 210$ px $\approx$ à l'échelle de l'essai.
tensor	"infinitesimal"	Type de tenseur de déformation. Options : "infinitesimal" (petites déformations), "green_lagrange" (grandes déformations), "eulerian_almansi".
compute_shear	True	Calcule et exporte la composante de cisaillement ε_{xy} .
compute_principal	True	Calcule les déformations principales $\varepsilon_1, \varepsilon_2$ et l'angle θ .
exclude_boundary	True	Exclut les nœuds du bord de la grille du calcul de déformation, où le voisinage circulaire reste incomplet.

4.8 Bloc UNCERTAINTY : incertitude

Paramètre	Valeur par défaut	Description
enabled	True	Active le module d'estimation de l'incertitude.
mode	"gum"	Mode de propagation. Le mode "gum" applique les formules analytiques du §2.6. Le mode "none" omet le calcul.

4.9 Bloc VIS : visualisation et export

Paramètre	Valeur par défaut	Description
dpi	180	Résolution des figures exportées en points par pouce.
export_format	"png"	Format d'export des figures. Options standard de Matplotlib : "png", "pdf", "svg".
smooth_method	"bspline"	Méthode de lissage pour le rééchantillonnage visuel des champs scalaires sur une grille régulière à haute résolution.
upsample_nx, upsample_ny	320	Résolution de la grille interpolée pour la visualisation du tableau de bord et des cartes de champs.
levels	140	Nombre de niveaux de couleur pour les cartes de contours.
quiver_skip	6	Facteur de sous-échantillonnage du champ vectoriel. Une flèche sur quiver_skip est affichée dans chaque direction.
quiver_gain	3.0	Facteur d'échelle des flèches du quiver.

4.10 Bloc RUNTIME : contrôle d'exécution

Paramètre	Valeur par défaut	Description
<code>selected_frame</code>	40	Indice de l'image DIC (base 0) pour laquelle des figures d'inspection détaillée sont générées à la fin du pipeline. L'indice d'image associé est <code>selected_frame + 1</code> .
<code>min_valid_points_per_frame</code>	300	Nombre minimal de nœuds avec corrélation réussie requis pour qu'une image soit considérée exploitable. Les images avec moins de points sont automatiquement écartées.
<code>trim_correlation_tail</code>	True	Active la troncature automatique des images finales dans lesquelles la corrélation s'effondre par perte du motif ou déplacement excessif.
<code>show_selected_frame_results</code>	True	Génère l'ensemble complet de figures d'inspection pour l'image sélectionnée.

5 Description technique des modules

5.1 Chargement des images : `image_loader.py`

La fonction `run_image_loader_v3()` effectue les opérations suivantes :

1. Recherche des fichiers dans `image_dir` correspondant à `image_pattern`, via `glob` avec tri alphanumérique.
2. Chargement de chaque image avec OpenCV ou PIL et conversion en niveaux de gris (`float32`, plage `[0, 1]`).
3. Vérification que toutes les images ont la même résolution. En cas d'incohérence, une exception est levée.
4. Stockage de la liste des images dans `project.images` et des métadonnées de la séquence dans `project.image_shape`, `project.num_images`.
5. Génération d'un aperçu avec la première, l'intermédiaire et la dernière image de la séquence (Figure 1).

L'image à l'indice `reference_index = 0` sert de référence pour toute l'analyse.

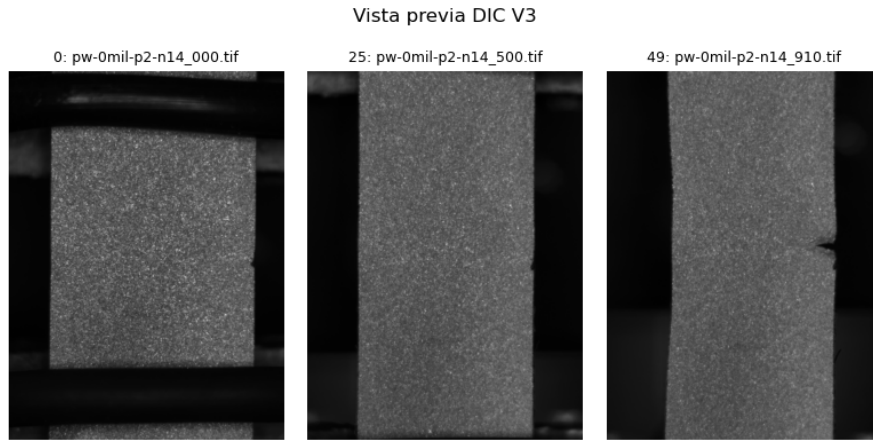


FIG. 1 : Aperçu de la séquence expérimentale généré par `image_loader.py`. Permet de vérifier la cohérence de l'éclairage, la résolution et la présence de déformation observable avant la corrélation.

5.2 Étalonnage : `calibration.py`

La fonction `run_calibration_v3()` exécute trois opérations en séquence :

1. **Chargement d'un étalonnage enregistré.** Recherche des fichiers `.npz` ou `.json` portant le nom `<project_name>_calibration_v3.*` dans le répertoire de sortie et dans le répertoire des images. S'il en trouve un valide et que `use_saved = True`, il le charge sans interaction de l'utilisateur.
2. **Étalonnage interactif d'échelle.** S'il n'existe pas d'étalonnage antérieur et que `interactive = True`, l'image de référence est affichée et l'utilisateur sélectionne deux points avec `ginput`. Après avoir saisi la distance réelle en mm dans la console, le système calcule :

$$\text{mm/px} = \frac{d_{\text{réelle}} [\text{mm}]}{d_{\text{px}} [\text{px}]}, \quad (11)$$

où $d_{\text{px}} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$.

3. **Correction de distorsion.** Si le fichier d'étalonnage contient `camera_matrix` (matrice intrinsèque 3×3) et `dist_coeffs` (coefficients de distorsion radiale/tangentielle), `cv2.undistort()` est appliqué à toute la séquence d'images.

L'étalonnage est automatiquement sauvegardé à la fin de l'étape aux formats `.npz` et `.json` pour réutilisation dans les exécutions ultérieures.

Lien avec le Manuel du participant

Le processus d'étalonnage de caméra par mire d'échiquier (Zhang, 2000), le facteur d'échelle et sa propagation d'incertitude sont décrits au chapitre 3 du Manuel du participant.

5.3 ROI et grille : `roi_grid.py`

La fonction `run_grid_definition_v3()` fonctionne de la manière suivante :

En mode interactif (`interactive = True`), elle affiche l'image de référence et demande à l'utilisateur de définir deux coins opposés du rectangle d'analyse. Le module applique automatiquement une marge interne de `subset_radius` pixels pour garantir que les subsets des nœuds du bord ne dépassent pas les limites de l'image.

Sur le rectangle ainsi défini, une grille orthogonale régulière est générée avec le pas (`spacing_x, spacing_y`). Les coordonnées des N nœuds sont stockées dans `project.grid_x` et `project.grid_y` sous forme de vecteurs de longueur N .

La figure résultante de la sélection de la ROI est affichée dans la console et peut être visualisée dans la Figure 2.

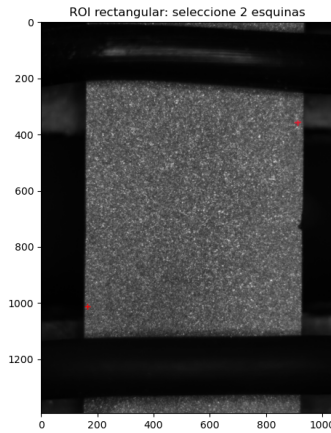


FIG. 2 : Sélection interactive de la ROI rectangulaire. Le contour vert délimite la région acceptée. Les nœuds de la grille sont placés à l'intérieur de ce contour, en respectant la marge de `subset_radius`.

5.4 Moteur de corrélation : `correlation_engine.py`

La fonction `run_correlation_v3()` implémente le cycle suivant pour chaque image $k = 1, \dots, N_{\text{images}}$ et chaque point de la grille $i = 1, \dots, N_{\text{pts}}$:

1. **Sélection des images.** En mode incrémental : $I_{\text{préc}} = I_k, I_{\text{cour}} = I_{k+1}$. En mode absolu : $I_{\text{préc}} = I_0, I_{\text{cour}} = I_{k+1}$.

2. **Extraction du template.** Le template de dimension $(2r+1)^2$ est extrait de $I_{\text{préc}}$ centré sur la position matérielle actuelle du point : $(x_0 + u_{k-1}, y_0 + v_{k-1})$.
3. **Prédiction.** La fenêtre de recherche est centrée sur le prédicteur : $(x_0 + u_{k-1} + \Delta u_{k-1}, y_0 + v_{k-1} + \Delta v_{k-1})$, où $\Delta u_{k-1}, \Delta v_{k-1}$ sont les incréments de l'image précédente.
4. **Matching ZNCC.** On évalue `cv2.matchTemplate(TM_CCOEFF_NORMED)` sur la fenêtre de recherche et on localise le maximum de la carte de corrélation.
5. **Raffinement sous-pixel.** Une interpolation parabolique 1D est appliquée au pic de la carte de corrélation (fonction `_parabolic_subpixel_1d`).
6. **Acceptation ou repli.** Si $C_{\text{ZNCC}} \geq \text{corrcoef_min}$, le point est accepté (`flag = 1`). Sinon, un repli en corrélation absolue est tenté en utilisant le template de l'image de référence globale (I_0) et une fenêtre de recherche élargie. Si le repli échoue également, le point est marqué invalide (`flag = 0`).
7. **Accumulation.** Les déplacements cumulés sont mis à jour : $u_k = u_{k-1} + \Delta u_k, v_k = v_{k-1} + \Delta v_k$.

Les matrices résultantes sont de dimension $N_{\text{pts}} \times N_{\text{images}}$: `project.corr_u`, `project.corr_v` (déplacements en px), `project.corr_coeff` (coefficient ZNCC), `project.corr_flags` (0 = invalide, 1 = valide).

V3 Corr 9/49 | cc=0.9315 | ok=2050/2050

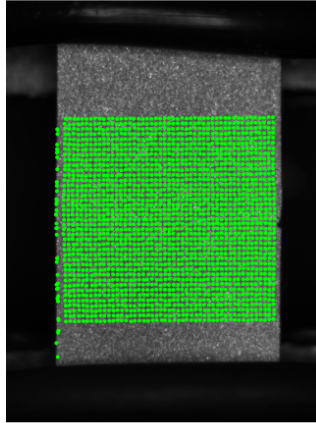


FIG. 3 : Contrôle visuel de la corrélation sur une image. Les nœuds verts représentent les points avec $C_{\text{ZNCC}} \geq \text{corrcoef_min}$; les rouges sont les points rejetés ou pour lesquels le repli a également échoué.

5.4.1 Troncature de la queue de corrélation

Après la corrélation, la fonction `_summarize_and_trim_correlation()` écarte les images finales dans lesquelles le nombre de points valides tombe en dessous de `min_valid_points_per_frame`. Ce mécanisme protège le calcul de la déformation contre les images à densité insuffisante de points, qui dégénèrent l'estimation des gradients.

5.5 Post-traitement du déplacement : `displacement.py`

La fonction `run_displacement_v3()` applique en séquence :

1. Détection des valeurs aberrantes par combinaison du seuil sur `corr_coeff` et du filtre médian sur les déplacements avec le seuil `median_threshold`.
2. Comblement des nœuds invalides par interpolation spatiale (`fill_method`).
3. Lissage spatial gaussien avec noyau `smoothing_kernel`.
4. Construction des champs exportables : `project.disp_x_px`, `project.disp_y_px`, et si un étalonnage valide existe, `project.disp_x_mm`, `project.disp_y_mm` et leurs modules.

Avant le calcul de la déformation, la fonction `_repair_bad_frames_temporal()` identifie les images avec un ratio de validité $< 0,50$ et remplace leurs déplacements par interpolation linéaire temporelle entre les images correctes voisines. Cela empêche que des images parasites contaminent la dérivation numérique de la déformation.

5.6 Calcul de la déformation : `strain.py`

La fonction `run_strain_v3()` implémente le flux décrit au §2.5. Les considérations d'implémentation pertinentes sont :

- Si des déplacements en mm existent (`disp_x_mm`, `disp_y_mm`) et que `mm_per_px` est valide, les gradients sont calculés en unités mm/mm (déformation adimensionnelle). Sinon, le calcul se fait en px/px, ce qui donne le même résultat adimensionnel si la grille est uniforme.
- Le paramètre `exclude_boundary = True` marque comme NaN les nœuds situés dans la bande de `strain_radius` nœuds du bord de la grille, où le voisinage circulaire serait incomplet.
- Les déformations principales ε_1 , ε_2 et l'angle θ sont calculés analytiquement à partir des composantes du tenseur au moyen des formules du cercle de Mohr.

5.7 Incertitude : `uncertainty.py`

La fonction `run_uncertainty_v3()` implémente le mode "gum" tel que décrit au §2.6. Les résultats stockés dans le projet sont :

- `project.ux`, `project.uy` : incertitude-type des déplacements $u(U)$ et $u(V)$, dans les mêmes unités que les déplacements (mm ou px).
- `project.u_combined` : incertitude combinée du vecteur de déplacement.
- `project.ustrain_xx`, `project.ustrain_yy`, `project.ustrain_xy` : incertitude des composantes du tenseur de déformation, adimensionnelles.

6 Exécution du pipeline

6.1 Prérequis système

- Python ≥ 3.9
- Bibliothèques : `numpy`, `scipy`, `opencv-python`, `matplotlib`, `pathlib` (standard).
- Optionnel : `torch` ≥ 1.10 pour l'accélération GPU en visualisation.
- Backend graphique : `Qt5Agg` (configuré dans `run_dic_V3.py`). `PyQt5` ou `PySide2` est nécessaire.

Important

Le script configure le backend de Matplotlib avec `matplotlib.use("Qt5Agg")` avant le premier `import matplotlib.pyplot`. Cela doit se produire avant toute autre importation de Matplotlib. Si l'environnement ne dispose pas de Qt5, changez cette ligne en `"TkAgg"` ou le backend disponible dans votre installation.

6.2 Structure de répertoires attendue

Listing 1 : Structure minimale avant l'exécution du pipeline.

```
1      projet/  
2      run_dic_V3.py  
3      DIC_2D_V3/  
4      __init__.py          # renommé depuis DIC_2D_V3__init__.py  
5      calibration.py  
6      config.py  
7      correlation_engine.py  
8      displacement.py  
9      image_loader.py  
10     project.py  
11     roi_grid.py  
12     strain.py  
13     uncertainty.py  
14     utils.py  
15     visualization.py  
16     images/  
17     pw-0mil-p2-n14_001.tif  
18     pw-0mil-p2-n14_002.tif  
19     ...
```

6.3 Exécution standard

Listing 2 : Exécution du pipeline depuis la racine du projet.

```
1 python run_dic_v3.py
```

Le flux effectif au sein de `main()` suit la séquence :

1. Création du projet `DICProjectV3` avec la configuration construite par `build_config()`.
2. Chargement des images avec `run_image_loader_v3()`.
3. Étalonnage d'échelle et correction optique avec `run_calibration_v3()`.
4. Définition de la ROI et génération de la grille avec `run_grid_definition_v3()`.
5. Corrélation incrémentale avec `run_correlation_v3()`.
6. Résumé de corrélation et troncature de la queue avec `_summarize_and_trim_correlation()`.
7. Post-traitement des déplacements avec `run_displacement_v3()`.
8. Réparation temporelle des images anormales avec `_repair_bad_frames_temporal()`.
9. Calcul de la déformation avec `run_strain_v3()`.
10. Estimation de l'incertitude avec `run_uncertainty_v3()`.
11. Génération des figures globales (dernière image valide).
12. Génération des figures de l'image sélectionnée (`selected_frame = 40`).
13. Sauvegarde du projet avec `proj.save()`.

7 Interprétation des sorties

7.1 Tableau de bord global

Le tableau de bord est généré par `plot_dashboard_v3()` pour la dernière image valide. Il présente six panneaux sur deux lignes (Figure 4) :

- Ligne supérieure : déplacements U (direction x), V (direction y) et module $|d| = \sqrt{U^2 + V^2}$, en mm si l'étalonnage est valide.
- Ligne inférieure : déformations ε_{xx} , ε_{yy} et ε_{xy} , mises à l'échelle en $\mu\epsilon$ si `strain_display_units = "microstrain"`.

Les champs sont rééchantillonnés sur une grille régulière de 320×320 points par interpolation B-spline pour améliorer l'apparence visuelle, mais les valeurs de couleur correspondent aux données calculées.

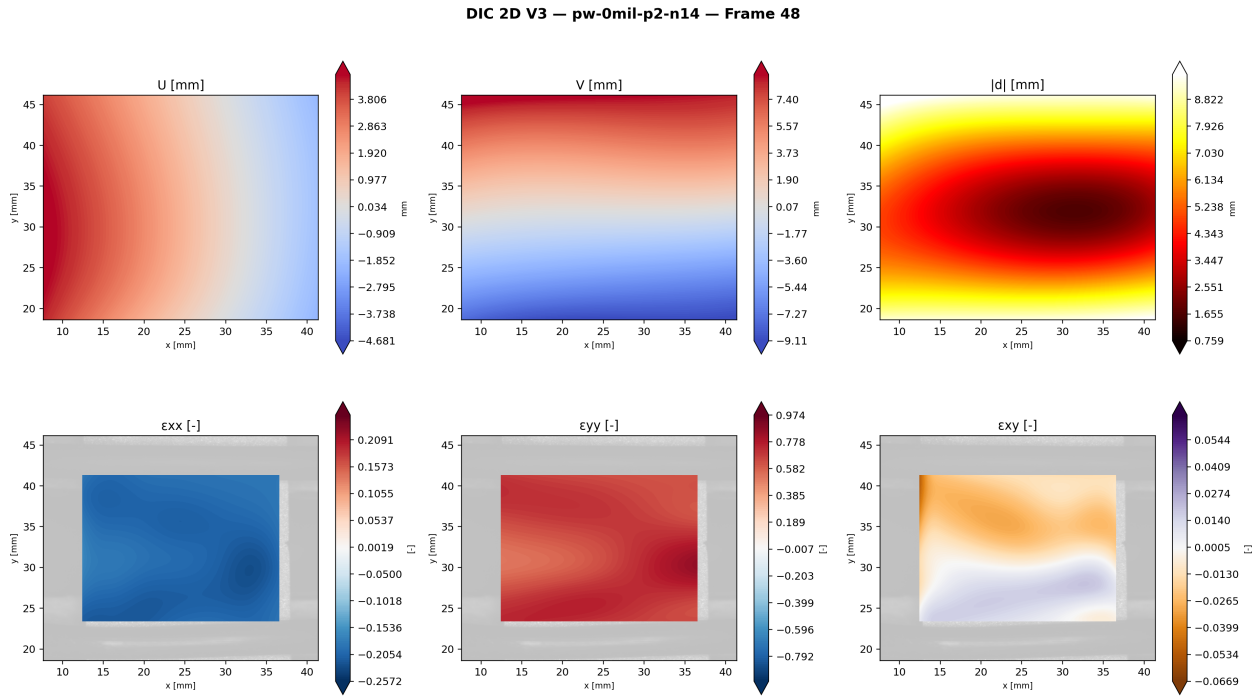


FIG. 4 : Tableau de bord global de la dernière image valide. Ligne supérieure : champs de déplacement U , V et module $|d|$. Ligne inférieure : composantes du tenseur de déformation ε_{xx} , ε_{yy} et ε_{xy} .

7.2 Champ vectoriel de déplacement (quiver)

La figure `quiver.png` présente le champ de déplacement sous forme de vecteurs (Figure 5). Chaque flèche représente le vecteur (U, V) en un nœud de la grille sous-échantillonné par `quiver_skip`. La couleur code le module $|d|$ et la longueur des flèches est mise à l'échelle par `quiver_gain`. Cette figure permet de vérifier la cohérence spatiale du champ cinématique et de détecter des discontinuités parasites ou des artefacts de corrélation.

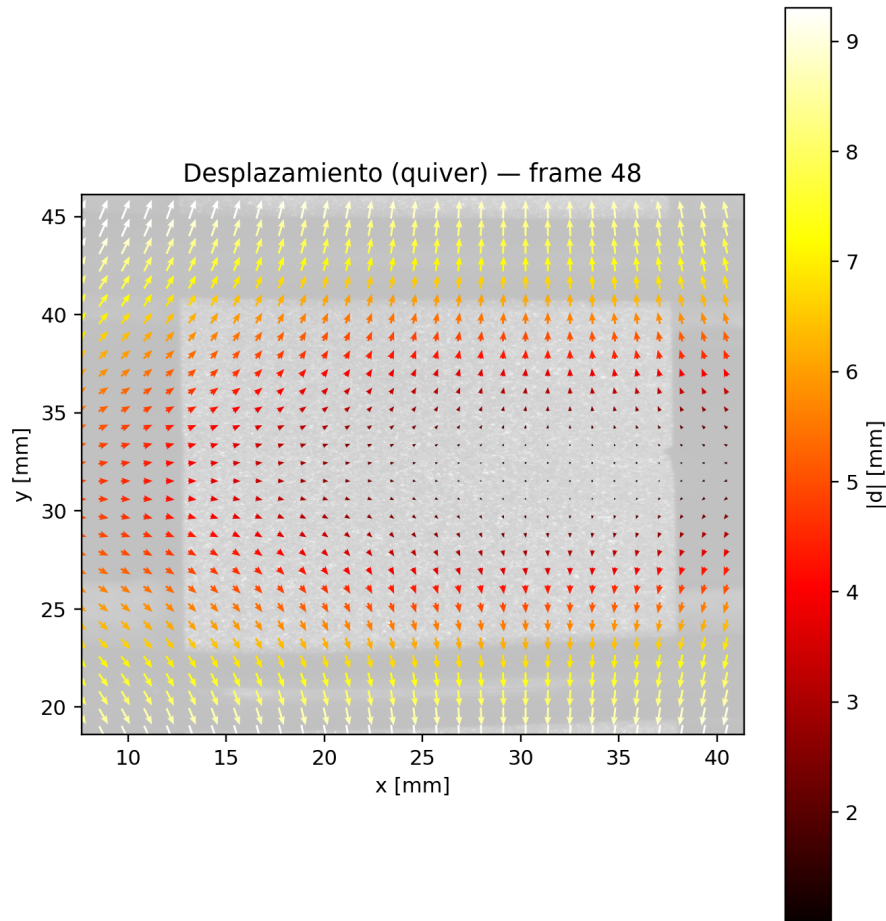


FIG. 5 : Champ vectoriel de déplacement. La direction de chaque flèche indique le sens du déplacement ; la couleur code le module $|d|$.

7.3 Cartes individuelles de déformation

Les fichiers `strain_xx.png`, `strain_yy.png` et `strain_xy.png` affichent chaque composante du tenseur séparément (Figure 6). L'échelle de couleur est symétrique par rapport à zéro et s'ajuste automatiquement aux percentiles [2, 98] pour éviter que des valeurs atypiques isolées ne saturent l'échelle chromatique.

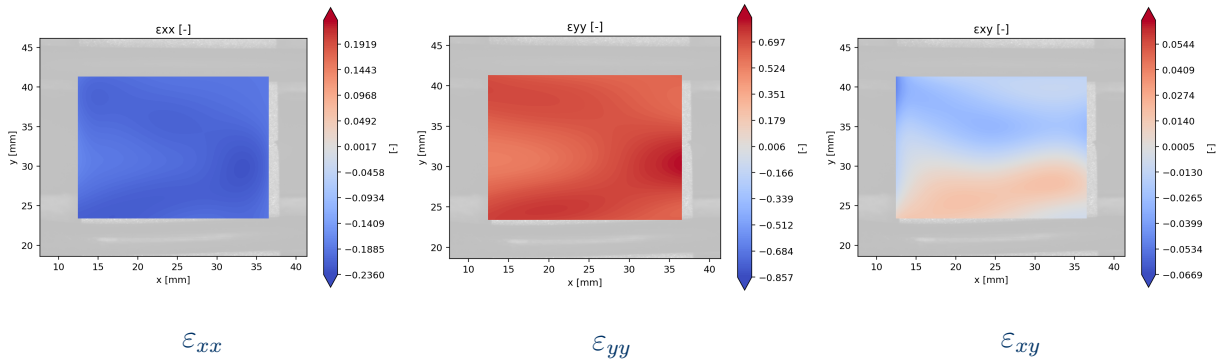


FIG. 6 : Cartes individuelles de déformation. ε_{xx} : déformation normale en x ; ε_{yy} : déformation normale en y ; ε_{xy} : composante de cisaillement. Lors d'un essai de traction uniaxiale, ε_{xx} doit dominer et ε_{yy} doit montrer une contraction par effet de Poisson.

7.4 Évolution temporelle de la déformation

La figure `strain_vs_frame.png` (Figure 7) montre la moyenne spatiale de chaque composante du tenseur en fonction de l'indice d'image. Cette courbe permet :

- D'identifier les étapes de chargement linéaire, non linéaire ou de décharge.
- De détecter les images avec corrélation dégradée (sauts brusques ne correspondant pas à des changements physiques réels).
- De vérifier que la troncature de la queue de corrélation s'est activée au bon moment.

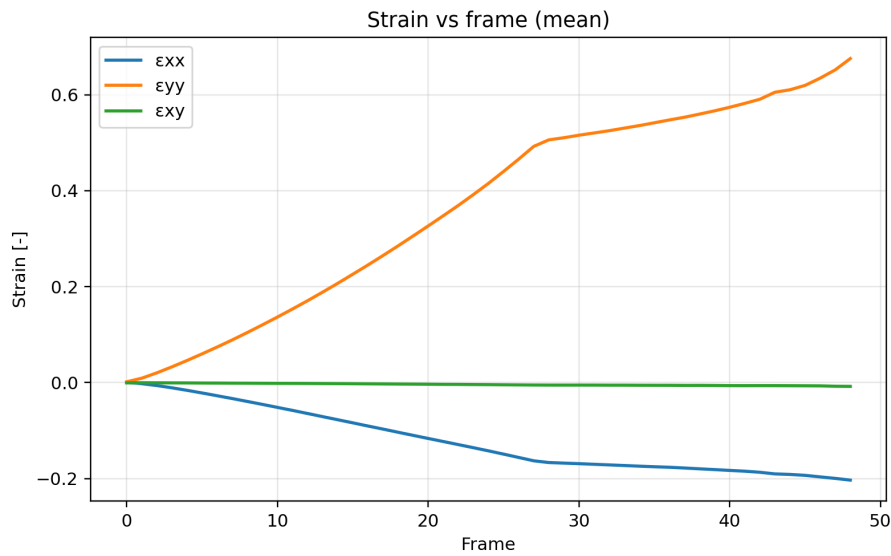


FIG. 7 : Évolution temporelle de la moyenne spatiale de ε_{xx} , ε_{yy} et ε_{xy} en fonction de l'image DIC.

7.5 Déformation vs. déplacement maximal

La figure `strain_vs_dmax.png` (Figure 8) relie la moyenne spatiale de ε_{xx} et ε_{yy} au déplacement maximal $|d|_{\max}$ de l'image correspondante. Cette représentation est utile pour détecter des non-linéarités globales entre la cinématique et la déformation, ou pour vérifier que la relation charge-déformation est cohérente avec le matériau et l'essai.

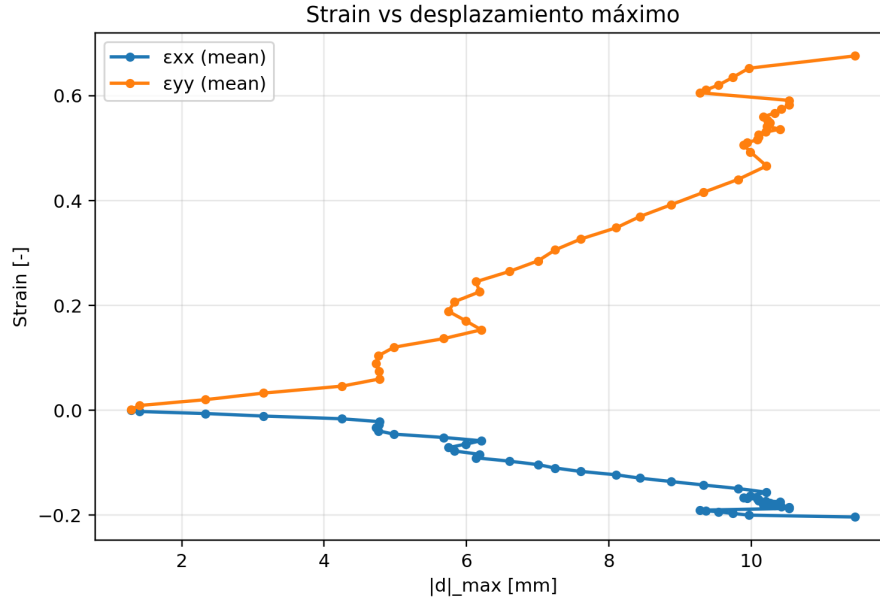


FIG. 8 : Relation entre la déformation moyenne ε_{xx} , ε_{yy} et le déplacement maximal $|d|_{\max}$ par image.

7.6 Résumé d'incertitude

La figure `uncertainty_summary.png` (Figure 9) présente la carte spatiale de l'incertitude-type des déplacements $u(U)$, $u(V)$, l'incertitude combinée u_c et l'incertitude du cisaillement $u(\varepsilon_{xy})$ pour la dernière image disponible.

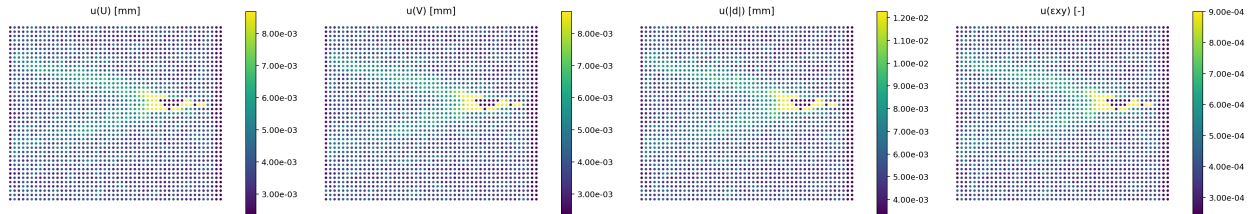


FIG. 9 : Résumé spatial de l'incertitude GUM. Ligne supérieure : $u(U)$, $u(V)$ et u_c dans les unités de déplacement actives. Ligne inférieure : $u(\varepsilon_{xy})$ adimensionnel.

L'incertitude de déplacement n'est pas uniforme : les nœuds avec un coefficient ZNCC bas reçoivent un facteur de pénalisation qui augmente leur incertitude estimée. Les zones avec un mouchetis dégradé ou les bords de l'éprouvette afficheront des valeurs plus élevées.

7.7 Tableau GUM

La figure `gum_table.png` (Figure 10) présente le bilan d'incertitude sous forme tabulaire : source d'incertitude, valeur numérique et contribution relative à l'incertitude combinée.

Presupuesto de incertidumbre GUM — pw-0mil-p2-n14

Incertidumbre de desplazamiento

Fuente de error	Tipo	Distribución	u_i (base)	c_i (sensib.)	Contribución $ c_i \cdot u_i $
Interpolación subpixel	B	Rectangular	0.0500 px	0.045911	2.2956e-03 mm
Ruido de imagen	A	Normal	-0.0150 px	0.045911	6.8867e-04 mm
Calidad de correlación	A	Normal	cc-penalty $\times \sigma$	variable	incl. en $u(U)$, $u(V)$
Calibración mm/px	B	Rectangular	2.2956e-04 mm/px	$ dj $	-2.2956e-04 mm
$u_c(U)$					4.0189e-03 mm
$u_c(V)$					4.0189e-03 mm
$u_c(d)$					5.6835e-03 mm

Propagación a strain (infinitesimal)

Fuente de error	Tipo	Distribución	u_i (base)	c_i (sensib.)	Contribución $ c_i \cdot u_i $
$u(U)$, $u(V)$ — gradiente	B	Propagación	$\sqrt{2} \cdot \sigma_u / L$	$1/L_x = 0.1037$	5.8950e-04
Strain radius $r=7$			$L_x = 2r \cdot \Delta x = 9.6414$ mm	$L_y = 9.6414$	
Grid spacing $\Delta x = 0.69$, $\Delta y = 0.69$ mm					
$u_c(\varepsilon_{xx})$					5.8950e-04
$u_c(\varepsilon_{yy})$					5.8950e-04
$u_c(\varepsilon_{xy})$					4.1684e-04

FIG. 10 : Tableau de bilan d'incertitude GUM généré par `plot_gum_table_v3()`.

8 Sorties de l'image sélectionnée

À la fin du pipeline global, le bloc `_show_selected_frame_results()` génère un ensemble de figures spécifiques pour l'image DIC d'indice `selected_frame = 40`. L'indice d'image correspondant est $40 + 1 = 41$.

Les figures générées pour cette image sont les mêmes que dans l'analyse globale (tableau de bord, quiver, cartes de ε_{xx} , ε_{yy} , ε_{xy}), avec les suffixes `_frame_040` dans les noms de fichier. De plus, l'image originale associée à l'image DIC et la carte de validité de corrélation pour cet instant sont affichées.

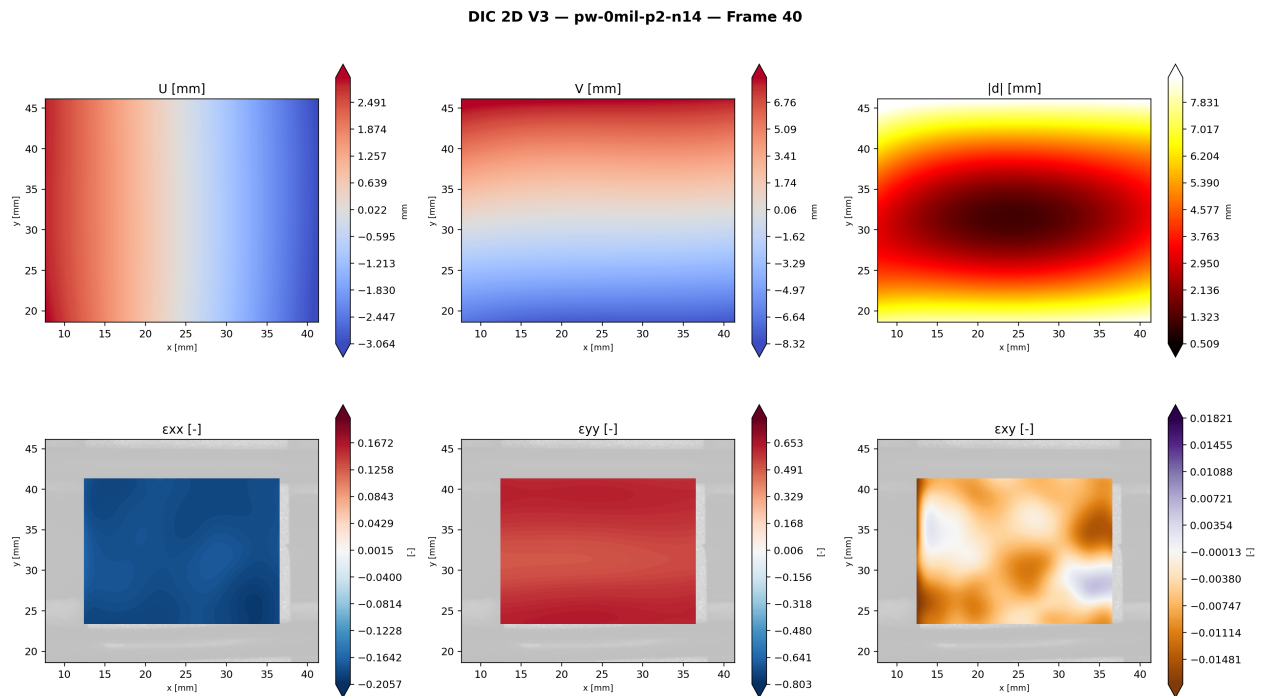


FIG. 11 : Tableau de bord d'inspection pour l'image sélectionnée (image DIC 40, image 41).

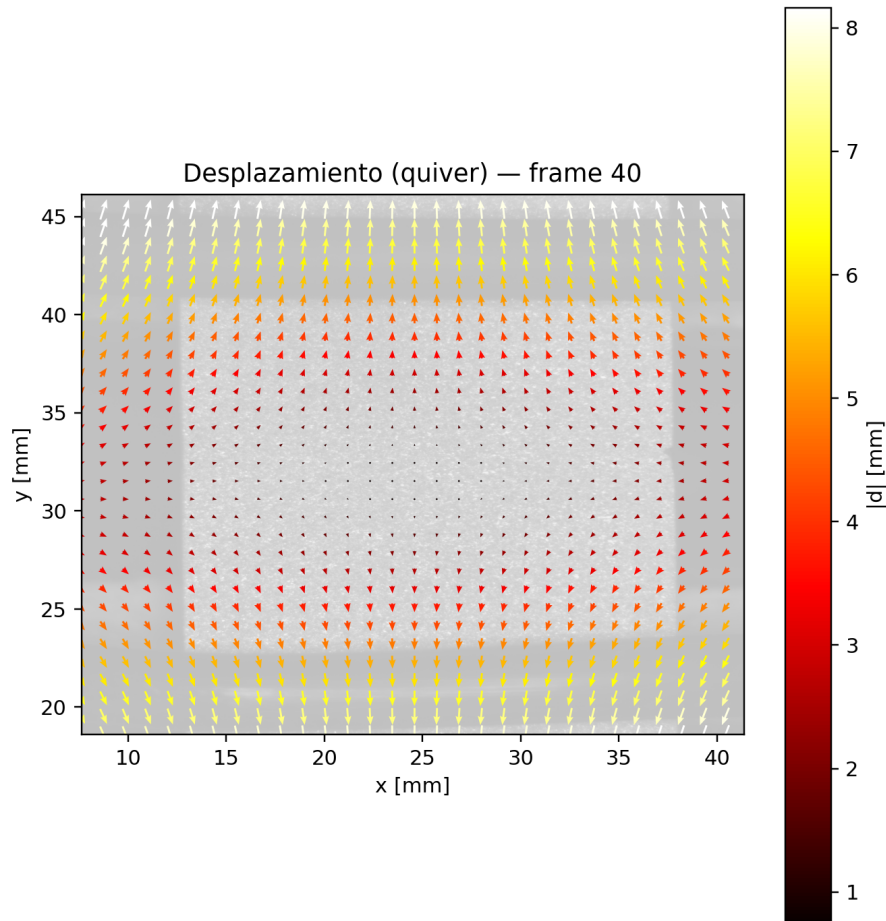


FIG. 12 : Champ vectoriel de déplacement pour l'image 40.

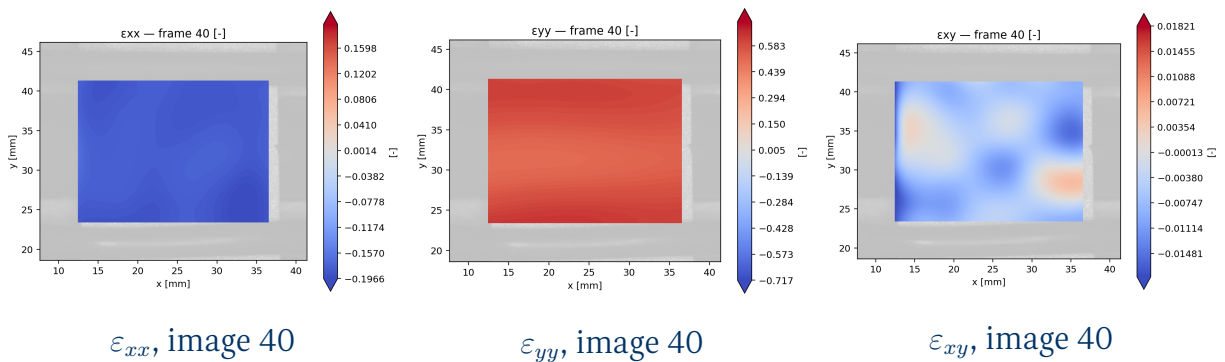


FIG. 13 : Cartes de déformation pour l'image sélectionnée. Elles permettent une inspection ponctuelle à tout instant de l'essai et de valider que la réparation temporelle des images anormales n'a pas introduit d'artefacts dans cette image spécifique.

9 Fichiers générés par le pipeline

Le répertoire `output_dir` contient trois catégories de sorties.

9.1 Fichiers d'étalonnage

- `<project_name>_calibration_v3.npz` : étalonnage au format binaire NumPy. Contient `mm_per_px`, `reprojection_error_px`, `stereo_rms_px`, `camera_matrix` et `dist_coeffs`.
- `<project_name>_calibration_v3.json` : le même contenu au format texte JSON, lisible sans Python.

9.2 Fichiers de données numériques

9.2.1 Déformation

- `strain_xx_v3.dat`, `strain_yy_v3.dat`, `strain_xy_v3.dat` : matrices $N_{pts} \times N_{images}$ au format texte délimité par tabulations, avec des valeurs au format exponentiel (`%.8e`).
- `strain_xx_v3_microstrain.dat`, etc. : mêmes matrices multipliées par 10^6 , lorsque `strain_display_units = "microstrain"`.

9.2.2 Incertitude

- `ux.dat`, `uy.dat` : incertitude-type de U et V .
- `u_combined.dat` : incertitude combinée du vecteur de déplacement.
- `ustrain_xx.dat`, `ustrain_yy.dat`, `ustrain_xy.dat` : incertitude des composantes du tenseur.

9.3 État du projet

- `<project_name>.dicv3` : fichier de sérialisation complète de l'objet `DICProjectV3`. Contient tous les tableaux de résultats et peut être récupéré avec `DICProjectV3.load()` pour une analyse postérieure sans ré-exécuter le pipeline.

10 Critères de configuration recommandés

Lien avec le Manuel du participant

Les plages quantitatives des paramètres présentés dans cette section correspondent au tableau de paramètres recommandés du §4.3.5 du Manuel du participant.

Paramètre	Effet principal	Plage typique	Critère de sélection
subset_radius	Résolution spatiale vs. bruit	10–40 px	≥ 3 –5 taches du motif
spacing_x/y	Densité de la grille	5–30 px	$\leq$ subset_radius
search_radius	Étendue de recherche	10–50 px	$>$ dépl. max. entre images
corrcoef_min	Seuil d'acceptation	0.4–0.7	Ajuster selon la qualité du motif
strain_radius	VSG effectif	3–15 nœuds	$L = 2r\Delta >$ échelle d'intérêt
median_threshold	Sensibilité aux aberrants	1.5–4.0	Plus faible = plus restrictif

TAB. 2 : Paramètres clés, leur effet et critères de sélection.

11 Limitations de la version actuelle

Les caractéristiques suivantes sont déclarées dans la configuration mais ne sont pas actives en V3.1 :

1. **Accélération GPU pour la corrélation.** Le paramètre `gpu_for_correlation` est présent mais le noyau NCC s'exécute sur CPU. L'accélération GPU est appliquée uniquement au rééchantillonnage en visualisation.
2. **Modèle de déformation affine du subset.** Le paramètre `warp = "affine"` est déclaratif. Le moteur actuel implémente la translation pure. L'extension au modèle affine (6 paramètres : u, u_x, u_y, v, v_x, v_y) est réservée aux versions futures.
3. **ROI polygonale interactive.** Le module `roi_grid.py` supporte la ROI polygonale programmatique, mais le flux par défaut utilise la ROI rectangulaire interactive.
4. **Convergence itérative IC-GN.** Les paramètres `cutoff_diffnorm` et `cutoff_iteration` sont prévus pour une implémentation future de l'optimiseur IC-GN (*Inverse Compositional Gauss-Newton*). Ils ne sont pas actifs dans le noyau NCC actuel.